Exercise01_Parallelizing

Parallelizing a basic linear algebra C++ program

This assignment for this lab class is to transform the given C++ into an equivalent CUDA program.
Specifically, all Matrix/Vector computations should take place in parallel on the GPU.

1. Run it, benchmark it and take a screenshot of the runtime of your first attempt.
2. Then, play with the kernel launch configuration (thread count, block count, etc.) to see how it impacts performance and take a screenshot of your best performing configuration.

Finally, hand in a ZIP containing your screenshots, CUDA code and Make file to build it, and the launch configurations you tried either in the form of comments in the CUDA code or as a standalone Markdown file.

1 Mekefile

build up executable file using Makefile

```
# Makefile for CUDA build
# Usage:
#   make
#   make run numElements=32768 threadsPerBlock=256
#   make clean

NVCC ?= nvcc
TARGET := slow
SRC := slow.cu
NVCCFLAGS ?= -O3 -std=c++20

$(TARGET): $(SRC)
    $(NVCC) $(NVCCFLAGS) -o $@ $<

run: $(TARGET)
    ./$(TARGET) $(numElements) $(threadsPerBlock)

clean:
    rm -f $(TARGET)
```

1.1 The number of registers used by a kernel

The number of registers used by a kernel can be found out using the --ptxas-options=-v nvcc command line option

```
(main) root@C.27659477:/workspace/yanbo-test$ make
nvcc -O3 -std=c++20 --ptxas-options=-v -o slow slow.cu
nvcc warning : Support for offline compilation for architectures prior to
'<compute/sm/lto>_75' will be removed in a future release (Use -Wno-
deprecated-gpu-targets to suppress warning).
ptxas info    : 0 bytes gmem
ptxas info    : Compiling entry function '_Z6matVecPKiS0_Pii' for 'sm_52'
ptxas info    : Function properties for _Z6matVecPKiS0_Pii
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 32 registers, used 1 barriers, 348 bytes cmem[0]
ptxas info    : Compile time = 24.197 ms
ptxas info    : Compiling entry function '_Z6vecAddPKiS0_Pii' for 'sm_52'
ptxas info    : Function properties for _Z6vecAddPKiS0_Pii
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 8 registers, used 0 barriers, 348 bytes cmem[0]
ptxas info    : Compile time = 1.232 ms
```

What the log says

- `--ptxas-options=-v` worked; you're compiling for **sm_52** (Maxwell).
- `matVec` **kernel**:
  - **32 registers/thread**, **1 barrier** (`__syncthreads()`), **no spills** (good).
  - Small constant memory (args) ~348 B.
- `vecAdd` **kernel**:
  - **8 registers/thread**, **no barriers**, **no spills**.
- `0 bytes gmem` = no *statically allocated* global memory inside kernels (runtime `cudaMalloc` is separate).
- Deprecation warning: offline compilation for < sm_75 will be removed in the future. It's a warning, not an error.

Occupancy quick take (sm_52 + your numbers)

- `threadsPerBlock = 256` , `matVec` uses 32 regs/thread.
- Registers per block = 256 × 32 = **8192**.
- On typical SM 5.2 (64k regs, 2048 threads max/SM), you can fit **8 blocks/SM** (8192×8=65536 regs) and hit **2048 threads/SM** ⇒ **~100% occupancy** by regs/threads.
- Shared memory per block is ~2 KB (for the reduction), not a limiter.

So 256 threads/block is a solid starting point. Still benchmark 128/256/512/1024.

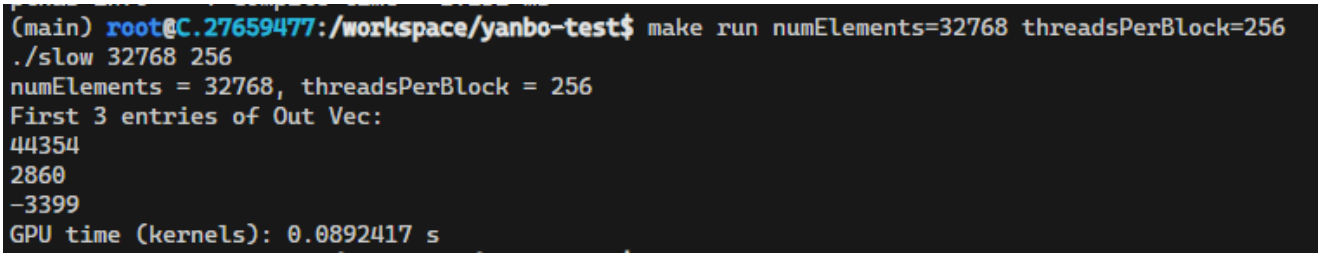
2 Experiment

2.1 First attempt

using

- numElements=32768
- threadsPerBlock=256

```
(main) root@C.27659477:/workspace/yanbo-test$ make run numElements=32768
threadsPerBlock=256
./slow 32768 256
numElements = 32768, threadsPerBlock = 256
First 3 entries of Out Vec:
44354
2860
-3399
GPU time (kernels): 0.0892417 s
```



2.2 The optimal kernel launch configuration

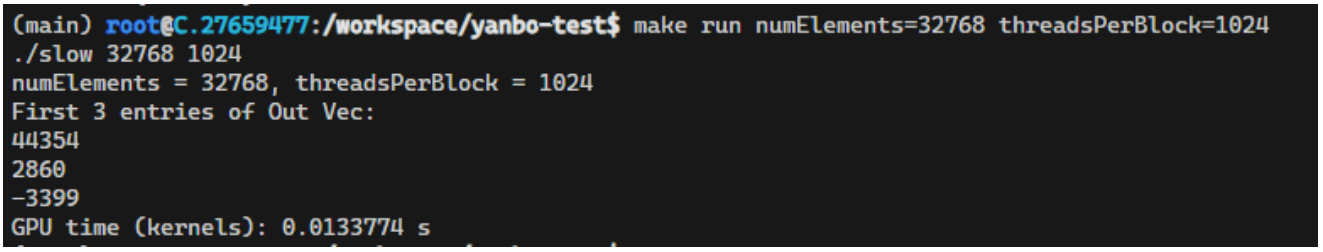
==The optimal kernel launch configuration is `threadsPerBlock =1024` ==

numElements=32768

threadsPerBlock	Time Consuming (second)
32	0.0136285
64	0.013816
128	0.0134465
256	0.015597
512	0.0140504
768	0.0162392
1024	0.0133774
1536	
2048	
4096	
6144	
8192	
16384	
32768	

When threadsPerBlock = 1024, It returns

```
(main) root@C.27659477:/workspace/yanbo-test$ make run numElements=32768
threadsPerBlock=1024
./slow 32768 1024
numElements = 32768, threadsPerBlock = 1024
First 3 entries of Out Vec:
44354
2860
-3399
GPU time (kernels): 0.0133774 s
```



When threadsPerBlock > 1024, It returns

```
First 3 entries of Out Vec :
0
0
0
```

It means the result of computation become unfeasible, when threadsPerBlock > 1024.

The maximal available threadsPerBlock number is 1024

3 Source Code slow.cu

```
// slow.cu – simple CUDA version of vector add + matrix-vector product
// Usage: ./slow [numElements] [threadsPerBlock]
// Defaults: numElements = 32768, threadsPerBlock = 256

#include <cuda_runtime.h>
#include <iostream>
#include <random>
#include <chrono>
#include <cstdint>

// Vector addition kernel
// Grid: grid1 = ( (numElements + threadsPerBlock - 1) / threadsPerBlock
```

```
)blocks.
// Block: threadsPerBlock threads.
// Each thread handles one element.
__global__ void vecAdd(const int32_t* a, const int32_t* b, int32_t* c, int
numElements) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < numElements)
        c[i] = a[i] + b[i];
}

/*
Matrix-vector multiplication kernel
Grid: grid2 = numElements (There is totally numElements block per grid. It
indicate that one block process one matrix row).
Block: block2 = threadsPerBlock.
Shared memory per block: threadsPerBlock * sizeof(long long) bytes.

Threads in the block sum this row with striding, then reduce.
Inside each block:
  row = blockIdx.x selects the current matrix row.
  Each thread does a strided partial dot product over that row:



Block-wide reduction in shared memory:
- Each thread writes localSum to sharedData[tid].
- A simple power-of-two reduction halves the active threads each step until
tid == 0 holds the row's dot product.
- Thread 0 writes out[row].



This design is easy to read: "one row per block, block threads cooperate to
sum the row."
*/

__global__ void matVec(const int32_t* __restrict__ mat,
                       const int32_t* __restrict__ vec,
                       int32_t* __restrict__ out,
                       int numElements) {
    int row = blockIdx.x;
    int tid = threadIdx.x;
    int threadsPerBlock = blockDim.x;

    if (row >= numElements)
        return;

    // Each thread does a strided partial dot product over that row:
    // Using 64-bit long long reduces overflow risk during accumulation
(final cast back to int32_t matches your original output type).
    long long localSum = 0;
    for (int j = tid; j < numElements; j += threadsPerBlock)
```

```
        localSum += (long long)mat[row * numElements + j] * (long
long)vec[j];

    extern __shared__ long long sharedData[];
    sharedData[tid] = localSum;
    __syncthreads();

    // reduction
    for (int step = threadsPerBlock >> 1; step > 0; step >>= 1) {
        if (tid < step)
            sharedData[tid] += sharedData[tid + step];
        __syncthreads();
    }

    if (tid == 0)
        out[row] = (int32_t)sharedData[0];
}

static void init_host(int numElements, int32_t* a, int32_t* b, int32_t* mat)
{
    std::mt19937 rng(2024);
    std::uniform_int_distribution<int32_t> dist(-16, 16);
    for (int i = 0; i < numElements; ++i) {
        a[i] = dist(rng);
        b[i] = dist(rng);
    }
    for (long long i = 0; i < 1LL * numElements * numElements; ++i)
        mat[i] = dist(rng);
}



/*
variable tpb: threads per block
numElements: size of vectors (n) and matrix (n x n)
*/
int main(int argc, char** argv) {
    int numElements = (argc >= 2 ? std::stoi(argv[1]) : 32768);
    int threadsPerBlock = (argc >= 3 ? std::stoi(argv[2]) : 256);
    if (threadsPerBlock <= 0)
        threadsPerBlock = 256;

    std::cout << "numElements = " << numElements
              << ", threadsPerBlock = " << threadsPerBlock << "\n";

    // host memory
    // Vectors: numElements * 4 bytes each (because int32_t).
    // Matrix: numElements * numElements * 4 bytes.
    size_t vectorBytes = sizeof(int32_t) * (size_t)numElements;
    size_t matrixBytes = sizeof(int32_t) * (size_t)numElements *
(size_t)numElements;
```

```
    int32_t *h_a=nullptr, *h_b=nullptr, *h_mat=nullptr, *h_out=nullptr;

    try {
        h_a   = new int32_t[numElements];
        h_b   = new int32_t[numElements];
        h_mat = new int32_t[(size_t)numElements * (size_t)numElements];
        h_out = new int32_t[numElements];
    } catch (...) {
        std::cerr << "Host memory allocation failed. Try smaller
numElements.\n";
        return 1;
    }

    init_host(numElements, h_a, h_b, h_mat);

    // device memory
    int32_t *d_a=nullptr, *d_b=nullptr, *d_tmp=nullptr, *d_mat=nullptr,
*d_out=nullptr;
    if (cudaMalloc(&d_a, vectorBytes) ||
        cudaMalloc(&d_b, vectorBytes) ||
        cudaMalloc(&d_tmp, vectorBytes) ||
        cudaMalloc(&d_out, vectorBytes) ||
        cudaMalloc(&d_mat, matrixBytes)) {
        std::cerr << "Device memory allocation failed. Try smaller
numElements.\n";
        return 1;
    }

    // copy to device
    cudaMemcpy(d_a, h_a, vectorBytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, vectorBytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_mat, h_mat, matrixBytes, cudaMemcpyHostToDevice);

    // kernel timing
    // Timing: CUDA events measure GPU time only (from right before vecAdd
to right after matVec).
    // Host⇔Device copies are not included in that time; you could measure
end-to-end with std::chrono if you need total wall time.
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start);

    // kernel 1: tmp = a + b
    dim3 grid1((numElements + threadsPerBlock - 1) / threadsPerBlock);
    dim3 block1(threadsPerBlock);
    vecAdd<<<grid1, block1>>>(d_a, d_b, d_tmp, numElements);

    // kernel 2: out = mat * tmp
    dim3 grid2(numElements);  // one block per matrix row
```

```
    dim3 block2(threadsPerBlock);
    size_t sharedBytes = threadsPerBlock * sizeof(long long);   // Shared
memory per block: threadsPerBlock * sizeof(long long) bytes.
    matVec<<<grid2, block2, sharedBytes>>>(d_mat, d_tmp, d_out,
numElements);

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float kernelMs = 0.0f;
    cudaEventElapsedTime(&kernelMs, start, stop);

    // copy back
    cudaMemcpy(h_out, d_out, vectorBytes, cudaMemcpyDeviceToHost);

    std::cout << "First 3 entries of Out Vec:\n";
    for (int i = 0; i < 3 && i < numElements; ++i)
        std::cout << h_out[i] << "\n";

    std::cout << "GPU time (kernels): " << (kernelMs / 1000.0) << " s\n";

    // cleanup
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_tmp); cudaFree(d_mat);
cudaFree(d_out);
    delete[] h_a; delete[] h_b; delete[] h_mat; delete[] h_out;

    return 0;
}
```